

Deep Learning

A Structured Guide Through Andrew Ng’s Deep Learning Specialization —
Theory, Code, Free Alternatives & Best Practices

Compiled as an MIT-style course pack on AI/ML/DL & NLP

August 2026

Contents

1	About This Guide & How to Use the Free Alternatives	3
2	Course 1: Neural Networks and Deep Learning (4 Weeks)	4
2.1	Week 1 — Introduction to Deep Learning	4
2.2	Week 2 — Neural Network Basics (Logistic Regression as a Neural Network) . .	4
2.3	Week 3 — Shallow Neural Networks	5
2.4	Week 4 — Deep Neural Networks	5
2.5	Free Alternatives — Course 1	6
3	Course 2: Improving Deep Neural Networks — Hyperparameter Tuning, Regularization & Optimization (3 Weeks)	7
3.1	Week 1 — Practical Aspects of Deep Learning	7
3.2	Week 2 — Optimization Algorithms	7
3.3	Week 3 — Hyperparameter Tuning, Batch Normalization & Programming Frameworks	8
3.4	Free Alternatives — Course 2	9
4	Course 3: Structuring Machine Learning Projects (2 Weeks)	10
4.1	Week 1 — ML Strategy (I)	10
4.2	Week 2 — ML Strategy (II)	10
4.3	Free Alternatives — Course 3	11
5	Course 4: Convolutional Neural Networks (4 Weeks)	12
5.1	Week 1 — Foundations of CNNs	12
5.2	Week 2 — Deep Convolutional Models: Case Studies	12
5.3	Week 3 — Object Detection	13
5.4	Week 4 — Special Applications: Face Recognition & Neural Style Transfer . . .	13
5.5	Free Alternatives — Course 4	14
6	Course 5: Sequence Models (4 Weeks)	15
6.1	Week 1 — Recurrent Neural Networks	15
6.2	Week 2 — NLP & Word Embeddings	15
6.3	Week 3 — Sequence-to-Sequence Models & Attention	15
6.4	Week 4 — Transformers	15
6.5	Free Alternatives — Course 5	15
7	Full Resource Summary Table	17
7.1	Lecture Alternatives (Free, Video) — Official Course Playlists	17

7.2	Lecture Alternatives (Free, Video) — Supplementary/Deeper Companions	17
7.3	Lab / Hands-On Alternatives (Free, Code)	17
8	Quick Revision Notes & Interview Q&A	19
8.1	One-Line Recall Sheet	19
8.2	Interview-Style Q&A	19

1 About This Guide & How to Use the Free Alternatives

This guide follows the exact structure of Andrew Ng’s **Deep Learning Specialization** (DeepLearning.AI, Coursera) — five courses, roughly 17 weeks of content — and expands every course into study notes: intuition, math, code, and best practices. For every course it lists a **free lecture alternative** (the actual official lecture videos, free on YouTube) and a **free lab alternative** (a place to run the code, since Coursera’s own graded labs sit behind its paid environment).

The Specialization can be audited for free on Coursera itself (video lectures + most content, no certificate/graded labs) — that’s always the zero-cost first option. What’s special about this Specialization specifically: DeepLearning.AI has uploaded the **complete, actual lecture videos** — not summaries or third-party reproductions — to its official YouTube channel, course by course. That means the free alternative below isn’t really an “alternative” at all — it’s the same content Coursera charges a subscription for, minus the graded quizzes/programming assignments and the certificate.

Resource type	Best free stand-in
Full lecture video replacement	Official DeepLearningAI YouTube playlists , one per course (all 5 linked below, real lecture content)
Hands-on coding practice	Kaggle Learn — free, browser-based, no setup: https://www.kaggle.com/learn
Notebook environment (GPU) for any code in this guide	Google Colab (free GPU/TPU notebooks): https://colab.research.google.com
Community-solved official labs (reference only)	GitHub repo aggregating notes + solved programming assignments for all 5 courses: https://github.com/amanchadha/coursera-deep-learning-specialization

Honest note before you switch platforms: the actual graded programming assignments (built cat-classifier, ResNet, YOLO, trigger-word detection, Transformer NMT, etc.) only exist inside Coursera’s own lab environment — there’s no free public mirror of *those exact* notebooks (and reproducing someone else’s solution defeats the point). The practical workaround used throughout this guide: rebuild the same *concept* (e.g., “build a CNN image classifier,” “build a character-level RNN”) using free datasets on Kaggle/Colab — you’ll write the code yourself, which is arguably better practice anyway.

2 Course 1: Neural Networks and Deep Learning (4 Weeks)

2.1 Week 1 — Introduction to Deep Learning

2.1.1 What's Driving the Deep Learning Rise?

Three converging trends: **data** (the internet generated enormous labeled datasets), **compute** (GPUs/TPUs made training large networks feasible), and **algorithms** (ReLU, better initialization, and architectural innovations made deeper networks trainable). Ng's often-cited "scale drives performance" idea: for large enough neural networks, more data and more compute reliably keep improving performance, unlike many classical ML algorithms that plateau.

2.1.2 Structured vs. Unstructured Data

Type	Examples	Best-suited models
Structured	Database tables, spreadsheets	Classical ML, tree ensembles
Unstructured	Images, audio, raw text	Deep neural networks

2.2 Week 2 — Neural Network Basics (Logistic Regression as a Neural Network)

2.2.1 Logistic Regression, Reframed as a One-Layer Network

Ng's pedagogical device: teach logistic regression using neural-network notation and computation-graph thinking (forward pass → cost → backward pass) *before* introducing hidden layers — so the jump to a real neural network in Week 3 is just "stack this same unit multiple times."

$$\hat{y} = \sigma(w^T x + b), \quad \mathcal{L}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

2.2.2 Vectorization

Replacing explicit for loops over training examples with matrix operations (NumPy) — critical for both speed and code clarity in every subsequent week.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def propagate(w, b, X, Y):
    m = X.shape[1]
    A = sigmoid(np.dot(w.T, X) + b) # forward pass, fully vectorized
    cost = -np.sum(Y * np.log(A) + (1 - Y) * np.log(1 - A)) / m
    dw = np.dot(X, (A - Y).T) / m # backward pass
    db = np.sum(A - Y) / m
    return dw, db, cost
```

2.2.3 Broadcasting

NumPy's broadcasting rules let you add/multiply arrays of different shapes without explicit loops (e.g., adding a bias vector $(n, 1)$ to every column of a (n, m) matrix) — a small mechanical detail that underlies almost every efficient deep learning implementation.

2.3 Week 3 — Shallow Neural Networks

2.3.1 A Single Hidden Layer Network

$$z^{[1]} = W^{[1]}x + b^{[1]}, \quad a^{[1]} = g^{[1]}(z^{[1]}), \quad z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}, \quad a^{[2]} = g^{[2]}(z^{[2]})$$

2.3.2 Why Non-Linear Activation Functions Are Essential

If every activation were linear, any depth of network collapses mathematically into a single linear transformation — no matter how many layers you stack, the whole network could only ever compute linear functions of the input. Non-linear activations (sigmoid, tanh, ReLU) are what give depth its expressive power.

2.3.3 Random Initialization

Initializing all weights to zero makes every neuron in a layer compute the exact same function (“symmetry”), so they never differentiate — weights must be initialized **randomly** (small random values) to break symmetry.

2.4 Week 4 — Deep Neural Networks

2.4.1 Forward and Backward Propagation, Generalized to L Layers

The same two-line forward/backward pattern from Week 2 repeats for every layer $l = 1 \dots L$, with each layer's output feeding the next — implemented cleanly via a for loop over layers storing intermediate values (“cache”) for use in the backward pass.

```
def linear_forward(A_prev, W, b):
    Z = np.dot(W, A_prev) + b
    cache = (A_prev, W, b)
    return Z, cache

def L_model_forward(X, parameters, L):
    caches = []
    A = X
    for l in range(1, L):
        A_prev = A
        Z, linear_cache = linear_forward(A_prev, parameters[f'W{l}'], parameters[f'b{l}'])
        A = np.maximum(0, Z)  # ReLU for hidden layers
        caches.append((linear_cache, Z))
    ZL, linear_cache = linear_forward(A, parameters[f'W{L}'], parameters[f'b{L}'])
    AL = 1 / (1 + np.exp(-ZL))  # sigmoid for output layer
    caches.append((linear_cache, ZL))
    return AL, caches
```

2.4.2 Hyperparameters vs. Parameters

Parameters (W , b) are learned by gradient descent. **Hyperparameters** (learning rate, number of layers/units, activation choice, number of iterations) are set by *you* and control how parameters get learned — tuning them is itself an empirical, iterative process (Course 2 covers this systematically).

2.5 Free Alternatives — Course 1

- **Lecture alternative:** Official DeepLearningAI YouTube playlist, “Neural Networks and Deep Learning” (43 videos — the real Course 1 lectures, free): https://www.youtube.com/playlist?list=PLkDaE6sCZn6Ec-XTbcX1uRg2_u4xOEky0. For a from-scratch, code-along companion covering the exact same forward/backward-propagation material, Andrej Karpathy’s “Neural Networks: Zero to Hero” series is a widely recommended pairing: <https://www.youtube.com/playlist?list=PLAqhlrjkxbuWI23v9cThsA9GvCAUhRvKZ>
- **Lab alternative:** Kaggle Learn’s *Intro to Deep Learning* course rebuilds the same “layers → activations → training loop” pattern hands-on in free notebooks: <https://www.kaggle.com/learn/intro-to-deep-learning>. To specifically replicate the Week 4 “build a deep net from scratch in NumPy” assignment, open a blank Google Colab notebook (<https://colab.research.google.com>) and implement the `L_model_forward/backward` functions above against the free `sklearn.datasets.load_digits` or Kaggle’s Cat-vs-Dog datasets.

3 Course 2: Improving Deep Neural Networks — Hyperparameter Tuning, Regularization & Optimization (3 Weeks)

3.1 Week 1 — Practical Aspects of Deep Learning

3.1.1 Train / Dev / Test Splits in the Deep Learning Era

With very large datasets, the classic 60/20/20 split is often overkill — a 98/1/1 split may still leave the dev and test sets with plenty of absolute examples. **Golden rule:** dev and test sets must come from the same distribution.

3.1.2 Regularization: L2 and Dropout

Technique	How it works
L2 regularization (“weight decay”)	Adds $\frac{\lambda}{2m} \sum W^{[l]} _F^2$ to the cost — shrinks weights toward zero, simplifying the effective network
Dropout	Randomly “drops” (zeroes out) a fraction of neurons on each training iteration — forces the network to not rely too heavily on any single neuron, acting like training an ensemble of smaller networks

```
def dropout_forward(A, keep_prob):  
    D = (np.random.rand(*A.shape) < keep_prob).astype(int)  
    A = A * D  
    A = A / keep_prob          # "inverted dropout" — rescale so expected value is unchanged  
    return A, D
```

3.1.3 Other Regularization Tricks

- **Data augmentation:** flips, rotations, crops — cheaply “creates” more training data for images.
- **Early stopping:** stop training when dev-set error starts increasing, even though training error keeps falling.
- **Gradient checking:** numerically approximate gradients to verify a backprop implementation is bug-free — used only for debugging, never during actual training (too slow).

3.2 Week 2 — Optimization Algorithms

3.2.1 Mini-Batch Gradient Descent

Instead of computing gradients over the *entire* training set (batch GD, slow per step) or a *single* example (stochastic GD, noisy), split training data into mini-batches (commonly 32–512 examples) — the standard practice in virtually all modern deep learning.

3.2.2 Momentum, RMSprop & Adam

Optimizer	Idea
Momentum	Maintains an exponentially weighted moving average of past gradients, smoothing out oscillations and accelerating convergence in consistent directions
RMSprop	Divides the learning rate by a moving average of recent squared gradients — dampens oscillation in steep dimensions, speeds up progress in flat ones
Adam	Combines Momentum + RMSprop — the default, most widely used optimizer in modern deep learning

```
import torch
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, betas=(0.9, 0.999), eps=1e-8)
```

3.2.3 Learning Rate Decay

Gradually reducing the learning rate over epochs lets training take large steps early (fast progress) and small steps late (fine convergence near the minimum) — common schedules: step decay, exponential decay, cosine annealing.

3.3 Week 3 — Hyperparameter Tuning, Batch Normalization & Programming Frameworks

3.3.1 Hyperparameter Search Strategy

- Prefer **random search** over grid search — with many hyperparameters, random sampling explores the space far more efficiently than an exhaustive grid.
- Use a **coarse-to-fine** search: start broad, then zoom into the promising region.
- Sample on an appropriate **scale** — e.g., learning rate on a log scale (10^{-4} to 10^0), not linear.

3.3.2 Batch Normalization

Normalizes the inputs to each layer (not just the input layer) to have stable mean/variance during training — dramatically speeds up training of deep networks and provides a mild regularizing effect as a side benefit.

```
import torch.nn as nn
layer = nn.Sequential(
    nn.Linear(256, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
)
```

3.3.3 Softmax Regression & Deep Learning Frameworks

Generalizes binary logistic regression's output to C classes (§ tied back to Course 1 of the *Machine Learning Specialization*'s Week 2 material). The course closes by introducing TensorFlow as a framework that handles backpropagation automatically via autodiff, so you only need to define the forward computation graph.

3.4 Free Alternatives — Course 2

- **Lecture alternative:** Official DeepLearningAI YouTube playlist, “Improving Deep Neural Networks: Hyperparameter tuning, Regularization and Optimization”: <https://www.youtube.com/playlist?list=PLkDaE6sCZn6Hn0vK8co82zjQtt3T2Nkqc>. For a deeper mathematical treatment of Adam/RMSprop/momentum, Stanford CS231n's optimization lecture is a widely used free companion (see Course 4 section below for the full CS231n playlist link).
- **Lab alternative:** Kaggle Learn's *Intro to Deep Learning*, lesson “Dropout and Batch Normalization,” directly mirrors this course's Week 1 and Week 3 material hands-on: <https://www.kaggle.com/learn/intro-to-deep-learning>. For experimenting with optimizers/learning-rate schedules, Google Colab's free GPU runtime is the practical place to run repeated training sweeps quickly: <https://colab.research.google.com>

4 Course 3: Structuring Machine Learning Projects (2 Weeks)

This course is unusual — no programming assignments, just two weeks of “flight simulator” case-study decision-making on how to run an ML project like a technical lead.

4.1 Week 1 — ML Strategy (I)

4.1.1 Orthogonalization

The principle of tuning “one knob to fix one problem” rather than hyperparameters that simultaneously affect multiple things — e.g., use regularization specifically to fix overfitting (variance), rather than a knob that also messes with training-set fit.

4.1.2 Setting Up Your Goal

Concept	Meaning
Single-number evaluation metric	Combine multiple metrics (e.g., precision & recall) into one number (e.g., F1) so different models can be compared unambiguously
Satisficing vs. optimizing metrics	Optimize the one metric that matters most, subject to the others merely clearing an acceptable threshold (e.g., “maximize accuracy subject to running in <100ms”)
Train/dev/test distribution	Dev and test sets must reflect the distribution of data you actually care about in production, or you’ll optimize for the wrong target

4.1.3 Comparing to Human-Level Performance

Human-level performance is used as a proxy for **Bayes error** (the theoretical best possible error rate) — as long as a model is below human-level, there are established tools (bias reduction: bigger model, train longer; variance reduction: more data, regularization) to keep improving; the story changes once a model surpasses human performance.

4.2 Week 2 — ML Strategy (II)

4.2.1 Error Analysis

Manually examine a sample (~100) of misclassified dev-set examples and categorize the failure types by hand — reveals which single fix (e.g., “improve performance on blurry images”) would yield the biggest error reduction, avoiding wasted engineering effort on low-impact problems.

4.2.2 Mismatched Training and Dev/Test Set Distributions

When training data comes from a different distribution than dev/test (e.g., web-scraped images for training, phone-camera images for the real product), carve out a small “**train-dev**” **set** (same distribution as training, held out) to distinguish variance problems from data-mismatch problems.

4.2.3 Transfer Learning, Multi-Task Learning & End-to-End Deep Learning

Concept	Idea
Transfer learning	Reuse a network pretrained on a large dataset (e.g., ImageNet), replacing/retraining only the final layer(s) for a new, related, smaller-data task
Multi-task learning	Train one network to simultaneously predict several related labels (e.g., detect pedestrians, cars, and traffic lights from one output layer), sharing lower-layer features
End-to-end deep learning	Replace a traditional multi-stage pipeline with a single network learned directly from raw input to final output — powerful with enough data, but loses the interpretability/hand-engineered-prior advantages of a staged pipeline

4.3 Free Alternatives — Course 3

- **Lecture alternative:** Official DeepLearningAI YouTube playlist, “Structuring Machine Learning Projects”: https://www.youtube.com/playlist?list=PLkDaE6sCZn6E7jZ9sN_xHwSHOdjUxUW_b. There’s no true equivalent elsewhere — this is Ng’s own applied-ML-leadership experience distilled into two weeks, so the official free playlist above is genuinely the best (and only) source for this exact material.
- **Lab alternative:** Not applicable — this course is case-study/strategy based with no programming assignments in the original either. As a practical substitute, apply the error-analysis and orthogonalization framework directly to your own Kaggle competition submissions or Course 1/4/5 lab models — pick a model you’ve already trained, manually inspect 20–30 of its mistakes on <https://www.kaggle.com/code>, and categorize the failure patterns.

5 Course 4: Convolutional Neural Networks (4 Weeks)

5.1 Week 1 — Foundations of CNNs

5.1.1 The Convolution Operation

A small filter/kernel slides across the input, computing a dot product at each position — detects local patterns (edges, textures) regardless of where they appear in the image (**translation invariance**), and dramatically reduces the parameter count compared to a fully-connected layer over raw pixels.

$$\text{output size} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

where n = input size, p = padding, f = filter size, s = stride.

5.1.2 Pooling Layers

Max/average pooling downsamples feature maps (e.g., 2×2 max pooling halves height and width), reducing computation and providing a small amount of translation invariance — has no learnable parameters.

```
import torch.nn as nn

conv_block = nn.Sequential(
    nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
)
```

5.2 Week 2 — Deep Convolutional Models: Case Studies

Architecture	Key innovation
LeNet-5	Original CNN (1998) — conv + pooling + fully connected, for digit recognition
AlexNet	Deeper/wider LeNet with ReLU; the 2012 ImageNet win that kicked off the deep learning boom
VGG	Very deep (16–19 layers), uses only small 3×3 filters stacked repeatedly — simple, uniform design
ResNet	Introduces skip/residual connections ($a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$), allowing networks with 100+ layers to train successfully by letting gradients flow directly through shortcut paths
Inception (“GoogLeNet”)	Runs multiple filter sizes in parallel within one “Inception module” and lets the network learn which combination works best, instead of hand-picking one filter size per layer

5.2.1 Transfer Learning for Computer Vision

The dominant practical pattern: take a network pretrained on ImageNet, freeze most layers, and retrain only the last few layers (or just the classifier head) on your smaller target dataset.

```
import torchvision.models as models
model = models.resnet50(weights='IMAGENET1K_V2')
for param in model.parameters():
    param.requires_grad = False          # freeze pretrained layers
model.fc = nn.Linear(model.fc.in_features, num_classes)  # replace + train only the head
```

5.3 Week 3 — Object Detection

Concept	Idea
Sliding windows / Intersection over Union (IoU)	Classic detection approach; IoU measures overlap between predicted and true bounding boxes, used to judge/filter predictions
Non-max suppression	Removes duplicate overlapping bounding-box predictions for the same object, keeping only the highest-confidence one
YOLO (“You Only Look Once”)	Predicts all bounding boxes and class probabilities in a single forward pass over a grid — fast enough for real-time detection
R-CNN family	Two-stage: first propose candidate regions, then classify each — generally more accurate but slower than YOLO
U-Net	Encoder-decoder architecture with skip connections, the standard architecture for semantic segmentation (per-pixel classification), widely used in medical imaging

5.4 Week 4 — Special Applications: Face Recognition & Neural Style Transfer

5.4.1 Face Recognition — One-Shot Learning via Similarity

Rather than training a classifier per person, learn a similarity/distance function via a **Siamese network** trained with **triplet loss** — pulls embeddings of the same person’s face closer together and different people’s faces farther apart, so new people can be added without retraining.

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

5.4.2 Neural Style Transfer

Combines the *content* of one image with the *style* of another by optimizing a generated image’s pixels directly to minimize a weighted sum of a content-similarity loss and a style-

similarity loss (computed from Gram matrices of a pretrained CNN's intermediate activations).

5.5 Free Alternatives — Course 4

- **Lecture alternative:** Official DeepLearningAI YouTube playlist, “Convolutional Neural Networks”: <https://www.youtube.com/playlist?list=PLkDaE6sCZn6Gl29AoE31iwdVwSG-KnDzF>. The single most-cited free deep-dive alternative/companion for computer vision specifically is **Stanford’s CS231n: Convolutional Neural Networks for Visual Recognition**: https://www.youtube.com/playlist?list=PL3FW7Lu3i5Jsnh1rnUwq_TcylNr7EkRe6
- **Lab alternative:** Kaggle Learn’s *Computer Vision* course builds CNNs hands-on with free notebooks and real image datasets (including the classic Dogs-vs-Cats competition data): <https://www.kaggle.com/learn/computer-vision>. For object detection/segmentation practice specifically, Kaggle hosts numerous free datasets and starter notebooks searchable at <https://www.kaggle.com/datasets>; Google Colab’s free GPU is the best place to fine-tune a pretrained torchvision model end to end: <https://colab.research.google.com>

6 Course 5: Sequence Models (4 Weeks)

6.1 Week 1 — Recurrent Neural Networks

Covers vanilla RNNs, the vanishing-gradient problem, and **GRU/LSTM** as the fix, plus **Bidirectional RNNs** (process the sequence both forward and backward, useful whenever the full sequence is available at once) and **Deep RNNs** (stack multiple recurrent layers). *(Full derivations and PyTorch code for this material are covered in the companion NLP guide's §7 "Sequence Models: RNN, LSTM, GRU.")*

6.2 Week 2 — NLP & Word Embeddings

Covers Word2Vec (skip-gram, negative sampling), GloVe, and using pretrained embeddings for downstream tasks like sentiment classification; also covers **debiasing word embeddings** — removing unwanted associations (e.g., gender bias in occupation-related word vectors) learned from biased training corpora, via projecting out the bias direction. *(Full coverage in the companion NLP guide's §3.5 "Word Embeddings.")*

6.3 Week 3 — Sequence-to-Sequence Models & Attention

Covers encoder-decoder architectures, **beam search** (keeps the top- k most likely partial sequences at each decoding step, instead of only the single greedy best-next-word — produces noticeably better translations/captions than greedy decoding), **BLEU score** for evaluating generated text, and the **attention mechanism** as the fix for the encoder-decoder bottleneck. Closes with an application to trigger-word/speech recognition. *(Full coverage in the companion NLP guide's §8 "Seq2Seq Models & Attention" and §13.2 for BLEU.)*

```
# Beam search sketch
def beam_search_step(sequences, scores, model, beam_width=3):
    all_candidates = []
    for seq, score in zip(sequences, scores):
        next_probs = model.predict_next(seq)                # log-probabilities
        top_k = next_probs.argsort()[-beam_width:]
        for token in top_k:
            all_candidates.append((seq + [token], score + next_probs[token]))
    all_candidates.sort(key=lambda x: x[1], reverse=True)
    return all_candidates[:beam_width]
```

6.4 Week 4 — Transformers

Covers self-attention, multi-head attention, and the full Transformer architecture — the same architecture that underlies BERT, GPT, and every modern LLM. *(Full derivation, scaled-dot-product-attention math, and from-scratch PyTorch implementation in the companion NLP guide's §10 "The Transformer Architecture.")*

6.5 Free Alternatives — Course 5

- **Lecture alternative:** Official DeepLearningAI YouTube playlist, "Sequence Models": https://www.youtube.com/playlist?list=PLkDaE6sCZn6F6wUI9tvS_Gw1vaFAx6rd6. For the Transformer/attention material specifically, 3Blue1Brown's "Attention in transformers, visually explained" is the most-recommended free visual companion: <https://www.youtube.com/watch?v=0j86Y331850>.

https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi; for a from-scratch coding walkthrough, Andrej Karpathy's "Let's build GPT from scratch" is the standard free reference: <https://www.youtube.com/playlist?list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ>

- **Lab alternative:** HuggingFace's free transformers + datasets libraries (installable in any Google Colab notebook, no cost) are the most direct hands-on substitute for the graded NMT-with-attention/trigger-word assignments — fine-tune a small pretrained model on a free dataset from <https://huggingface.co/datasets> or <https://www.kaggle.com/datasets>. Kaggle Learn's *Natural Language Processing* course also covers embeddings and sequence classification hands-on: <https://www.kaggle.com/learn/natural-language-processing>

7 Full Resource Summary Table

7.1 Lecture Alternatives (Free, Video) — Official Course Playlists

Course	Covers	Link
Course 1	Neural Networks and Deep Learning (43 videos)	https://www.youtube.com/playlist?list=PLkDaE6sCZn6Ec-XTbcX1uRg2_u4xOEky0
Course 2	Improving Deep Neural Networks	https://www.youtube.com/playlist?list=PLkDaE6sCZn6Hn0vK8co82zjQtt3T2Nkqc
Course 3	Structuring Machine Learning Projects	https://www.youtube.com/playlist?list=PLkDaE6sCZn6E7jZ9sN_xHwSHOdjUxUW_b
Course 4	Convolutional Neural Networks	https://www.youtube.com/playlist?list=PLkDaE6sCZn6Gl29AoE31iwdVwSG-KnDzF
Course 5	Sequence Models	https://www.youtube.com/playlist?list=PLkDaE6sCZn6F6wUI9tvS_Gw1vaFAx6rd6

7.2 Lecture Alternatives (Free, Video) — Supplementary/Deeper Companions

Resource	Best for	Link
Andrej Karpathy — “Neural Networks: Zero to Hero”	From-scratch backprop, GPT, tokenizers, coded live	https://www.youtube.com/playlist?list=PLAqhIrkxibuWI23v9cThsA9GvCAUhRvKZ
Stanford CS231n	Deep dive into CNNs / computer vision	https://www.youtube.com/playlist?list=PL3FW7Lu3i5Jsnh1rnUwq_TcylNr7EkRe6
3Blue1Brown — Neural Networks & Attention	Visual, first-principles math for NN & Transformers	https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

7.3 Lab / Hands-On Alternatives (Free, Code)

Resource	Covers	Link
Kaggle Learn — Intro to Deep Learning	Layers, activations, dropout, batch norm, SGD	https://www.kaggle.com/learn/intro-to-deep-learning
Kaggle Learn — Computer Vision	CNNs, transfer learning, data augmentation	https://www.kaggle.com/learn/computer-vision

Resource	Covers	Link
Kaggle Learn — Natural Language Processing	Embeddings, sequence classification	https://www.kaggle.com/learn/natural-language-processing
HuggingFace transformers + datasets (free, on Colab)	Fine-tuning pretrained sequence/Transformer models	https://huggingface.co/datasets
Google Colab	Free GPU/TPU notebooks for any code in this guide	https://colab.research.google.com
GitHub — amanchadha/coursera-deep-learning-specialization	Reference notes + solved assignments for all 5 courses	https://github.com/amanchadha/coursera-deep-learning-specialization

8 Quick Revision Notes & Interview Q&A

8.1 One-Line Recall Sheet

Concept	One-liner
Why deep learning scales	More data + more compute keeps improving large neural nets, unlike most classical ML
Vectorization	Replace for-loops with matrix ops — essential for speed
Why non-linear activations matter	Without them, any depth of network collapses into one linear function
Dropout in one line	Randomly zero out neurons each iteration to prevent co-adaptation/overfitting
Adam optimizer	Momentum + RMSprop combined — the default choice
Batch normalization	Normalize each layer's inputs during training — trains deep nets faster and more stably
Orthogonalization	Use one control knob to fix one specific problem, not a knob with tangled effects
Satisficing vs. optimizing metric	Optimize one metric; the rest just need to clear a bar
ResNet's key idea	Skip/residual connections let gradients bypass layers, enabling very deep networks
YOLO in one line	Predict all boxes + classes in one forward pass — fast, real-time detection
Siamese network + triplet loss	Learn a distance function so new classes (faces) can be recognized without retraining
Beam search	Keep top-k candidate sequences at each decoding step instead of one greedy best guess
Transfer learning (vision)	Freeze a pretrained CNN's early layers, retrain only the last layers on your data

8.2 Interview-Style Q&A

Q. Why is Xavier/He initialization preferred over pure random initialization? A. Poorly scaled initial weights cause activations (and gradients) to shrink or explode as they pass through many layers. Xavier/He initialization scales the random weights based on the number of input/output units of a layer, keeping the variance of activations roughly constant across layers — critical for training deep networks successfully.

Q. Why does dropout act as a regularizer? A. By randomly removing neurons on each training pass, no single neuron can rely on any one other specific neuron being present — this discourages complex co-adaptations between neurons and has a similar effect to training and averaging many smaller networks (an implicit ensemble).

Q. What problem do residual (skip) connections solve in very deep networks? A. Without them, gradients tend to vanish as they backpropagate through many layers, and very deep “plain” networks can actually perform *worse* than shallower ones (the degradation problem) — not due to overfitting, but because they become harder to optimize. Skip connections give

gradients a direct shortcut path, making it easy for a layer to at least learn the identity function, so adding depth never has to hurt performance.

Q. What's the difference between a satisficing metric and an optimizing metric? A. You pick one metric to optimize (maximize/minimize as much as possible) and treat all others as satisficing — they just need to clear a fixed threshold. E.g., “maximize classification accuracy, subject to inference time under 100ms” — accuracy is optimizing, latency is satisficing.

Q. Why is transfer learning particularly effective in computer vision? A. Early CNN layers learn very general, broadly reusable features (edges, textures, simple shapes) regardless of the specific dataset they were trained on; only the later layers specialize to the specific classes of the original task. Freezing those general early layers and retraining just the later/final layers lets you leverage huge pretrained datasets (like ImageNet) even when your own target dataset is small.

Q. How does the attention mechanism address the encoder-decoder bottleneck in sequence-to-sequence models? A. A vanilla encoder-decoder compresses the entire input sequence into one fixed-length vector, which loses information on long sequences. Attention lets the decoder access a weighted combination of *all* encoder hidden states at each output step, so it can dynamically “look back” at the most relevant parts of the input instead of relying on a single bottlenecked summary.

End of document — structured to mirror Andrew Ng's Deep Learning Specialization course-by-course, with a free lecture and lab alternative attached to every course, for full-course revision or a completely free self-study path. See the companion “Machine Learning” and “Natural Language Processing” guides for the classical-ML and NLP-specific material this Specialization builds on and extends.